青岛创新发展基地

海洋运载器建模与操控 课程作业

学　　号： S324527045

专　　业：控制工程

学生姓名：李颖

任课教师：吴迪 教授

2024 年 12 月

目　　录

1　前言 1
2　水下航无人航行器简介 1
　2.1 结构 1
　2.2 控制流程 3
3　AUV 固有模型 3
　3.1 AUV 运动学模型： 4
　3.2 AUV 动力学模型 4
　3.3 AUV 推进系统建模与推力分配 6
　　3.3.1 推进系统模型 6
　　3.3.2 推力分配方法 7
4　控制器设计 8
　4.1 外环 MPC 运动学控制器设计 8
　　4.1.1 运动学方程线性化处理 8
　　4.1.2 系统的未来状态预测 9
　　4.1.3 AUV 系统预测控制器的约束优化 10
　4.2 内环变论域 S 平面动力学控制器设计 12
　　4.2.1 常规变论域模糊控制器 12
　　4.2.2 改进思路 12
　　4.2.3 针对过驱动 AUV 的 P - S - VUD FLC 设计过程 14
　4.3 推力分配控制器 14
5　程序仿真 15
　5.1 参数设置 15
　5.2 仿真结果 16
　　5.2.1 无扰动下的仿真结果 16
　　5.2.2 扰动下的仿真结果 17
6　程序说明 18
　6.1 main 函数 18
　6.2 参数模块 18
　6.3 控制器模块 19
　6.4 模型模块 21
　6.5 可视化模块 22

Three-Dimensional Path Tracking of Over-Actuated AUVs Based on MPC and Variable Universe S-Plane Algorithms

基于模型预测控制和变论域 S 平面算法的超驱动自主水下航行器三维路径跟踪

1 前言

本作业参考的文献为 Three-Dimensional Path Tracking of Over-Actuated AUVs Based on MPC and Variable Universe S-Plane Algorithms（基于模型预测控制和变论域 S - 平面算法的超驱动自主水下航行器三维路径跟踪）参考文献原文.pdf，该论文由中国石油大学（北京）信息科学与工程学院与海洋石油工程股份有限公司联合发表的，作者包括 Feng Xu、Lei Zhang 和 Jibin Zhong，并于 2024 年 2 月 27 日发表在 Journal of Marine Science and Engineering 期刊上。该论文介绍了超驱动自主水下航行器（AUV）的一种路径跟踪方法，其控制器分为外环 MPC 控制器、内环变论域 S -平面控制器以及机械装置推力输出模块。

2 水下航无人航行器简介

2.1 结构

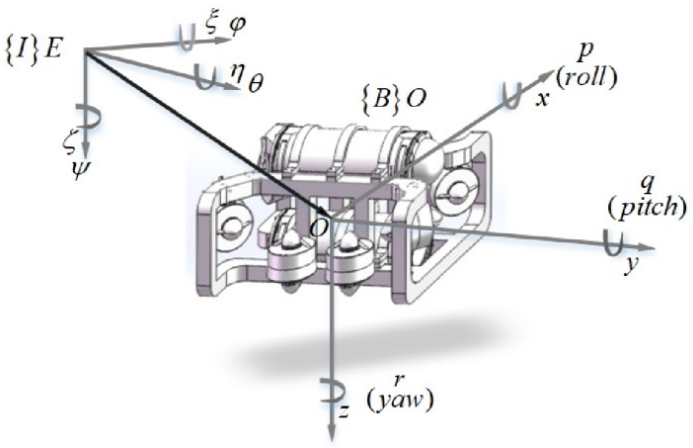

图 1 水下航行器实物图

上图为该论文建立的过驱动水下无人航行器模型，机械结构上具有较高的对

称性，执行机构包含 8 个推进器。其中参数为：

u,v,w 为航行器自身坐标系下三个方向的速度；

p,q,r 为航行器自身坐标下三个方向轴的角速度；

x,y,z 为航行器在北东地坐标系下三个方向前进的距离；

φ,θ,ψ为航行器自身坐标系下沿三个坐标轴旋转过的角度。

该航行器有 8 个螺旋桨，8 个螺旋桨空间位置关系如下表所示：

表 1 航行器推进器空间分布

编号	说明	x(m)	y(m)	z(m)	安装角度
1	水平弓左	*c*	*-b*	*m*	与 Ox 轴成 45° 角
2	水平弓右	*c*	*b*	*m*	与 Ox 轴成 45° 角
3	水平弓左	*-c*	*-b*	*m*	与 Ox 轴成 45° 角
4	水平弓右	*-c*	*b*	*m*	与 Ox 轴成 45° 角
5	垂直向左弯曲	*d*	*-f*	*n*	垂直于 xOy 平面
6	垂直向右弯曲	*d*	*f*	*n*	垂直于 xOy 平面
7	垂直向左弯曲	*-d*	*-f*	*n*	垂直于 xOy 平面
8	垂直向右弯曲	*-d*	*f*	*n*	垂直于 xOy 平面

其中，c=0.3，b=0.2，d=0.1，f=0.1，m=0.1，n=-0.2

8 个推进器的空间分布特点如下图所示：其中 1234 号推进器在一个平面上，分别与 x 轴成 45° 夹角；5678 号推进器在一个平面，与 z 轴平行。

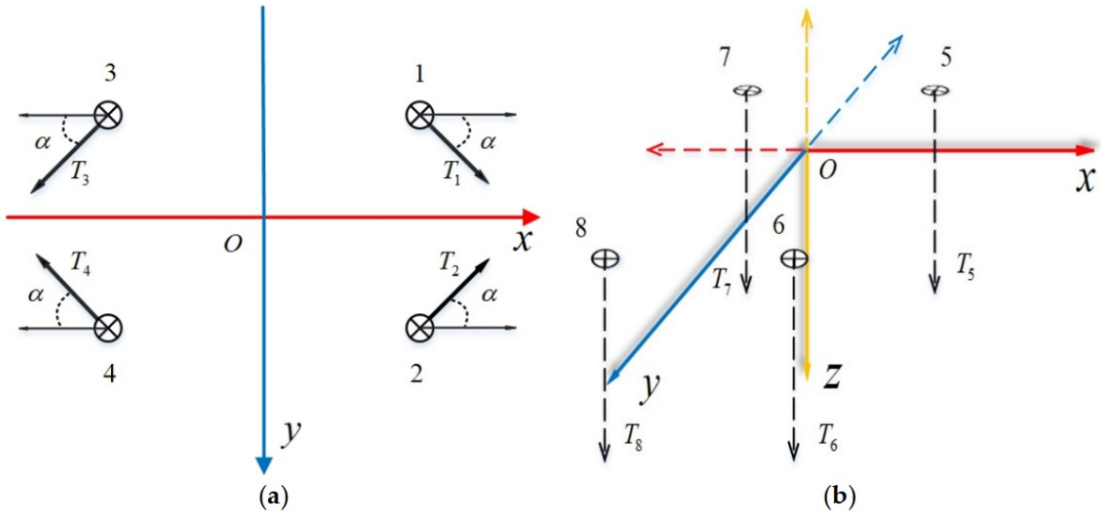

图 2 航行器推进器空间分布

2.2 控制流程

该水下无人航行器控制逻辑如下图所示：

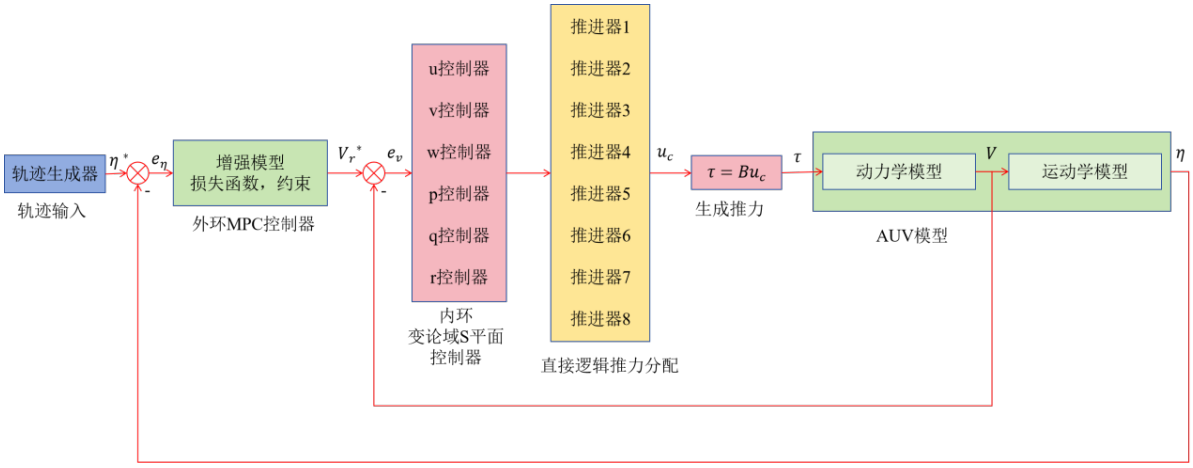

图 3 水下无人航行器控制流程

AUV 轨迹跟踪控制分为如下几个步骤：

①AUV 实时获得自身的位置姿态η和速度/角速度 V，并将其反馈给外环和内环控制器。

②内环控制器对输入的目标位置η^*和反馈的实际位置η进行处理，并对速度/角速度进行预测，进行控制预测和轨迹预测，输出最佳的期望速度V_r^*。

③内环控制器根据输入的期望速度V_r^*和当前的实际速度 V，进行处理，得到期望的抽象速度；

④将抽象速度输入给直接逻辑推力分配器，生成 8 个螺旋桨的转速。

⑤8 个螺旋桨的转速将会生成 3 个方向的推力和相对应的扭矩。将其输入给 AUV 动力学模型，AUV 就会在该推力和扭矩下进行相对应的运动，进而生成实时的 AUV 姿态和速度。

⑥将其记录下来，用于后续的绘制操作。

3 AUV 固有模型

AUV 固有模型有 kinematic 运动学模型和 dynamic 动力学模型。运动学模型就是已知 AUV 的速度、角速度信息，计算接下来的位置信息；动力学模型就是已知 AUV 上所受到的力，求得接下来 AUV 的速度、角速度信息。本论文中的 AUV 模型如下 3.1 和 3.2 所示：

3.1 AUV 运动学模型：

为了更好的描述 AUV 的模型，将η定义为 AUV 的空间位置姿态，V 定义为 AUV 的速度、角速度：

$$\begin{cases}\eta = [x, y, z, \varphi, \theta, \psi]^T \\ V = [u, v, w, p, q, r]^T\end{cases} \tag{1}$$

AUV 总体的运动学模型如下所示：

$$\dot{\eta} = J(\eta)V = \begin{bmatrix} J_1(\eta_2) & 0_{3\times3} \\ 0_{3\times3} & J_2(\eta_2)\end{bmatrix} V \tag{2}$$

其中：

$$\dot{\eta}_1 = J_1(\eta_2)V_1 \tag{3}$$

$$J_1(\eta_2) = \begin{bmatrix} \cos\theta\cos\psi & -\cos\varphi\sin\psi + \sin\varphi\sin\theta\cos\psi & \sin\varphi\sin\psi + \cos\varphi\sin\theta\cos\psi \\ \cos\theta\cos\psi & -\cos\varphi\cos\psi + \sin\varphi\sin\theta\sin\psi & -\sin\varphi\cos\psi + \cos\varphi\sin\theta\sin\psi \\ -\sin\theta & \cos\theta\sin\psi & \cos\varphi\sin\theta \end{bmatrix} \tag{4}$$

$$J_2(\eta_2) = \begin{bmatrix} 1 & \sin\varphi\tan\theta & \cos\varphi\tan\theta \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \dfrac{\sin\varphi}{\cos\theta} & \dfrac{\cos\varphi}{\cos\theta} \end{bmatrix} \tag{5}$$

上式中，$J(\eta)$为机体坐标系与惯性坐标系之间的变换矩阵，$0_{3\times3}$为 3×3 的零矩阵，$J_1(\eta_2)$为线速度变换矩阵，$J_2(\eta_2)$为角速度变换矩阵。根据上式，在已知 AUV 速度、角速度以及自身姿态的情况下，就可以求得接下来 AUV 在大地坐标系下的加速度，根据大地坐标系下的加速度就可求得位置、姿态信息。

3.2 AUV 动力学模型

AUV 总体的动力学模型如下所示：

$$M\dot{V} + C(V) + D(V)V + g(\eta) = \tau + \tau_E \tag{6}$$

上式中，M 为惯性矩阵，由刚体质量惯性矩阵M_{RB}和附加质量矩阵M_A组成。刚体质量惯性矩阵M_{RB}表示 AUV 运动过程中质量和转动惯量关于质心的分布情况。附加质量矩阵M_A表示由于 AUV 在运动过程中遇到的流体环境而产生的附加质量矩的响应。

$$M = M_{RB} + M_A \tag{7}$$

$$M_{RB} = {M_{RB}}^T = \begin{bmatrix} m & 0 & 0 & 0 & mz_g & my_g \\ 0 & m & 0 & -mz_g & 0 & mx_g \\ 0 & 0 & m & my_g & -mx_g & 0 \\ 0 & -mz_g & my_g & I_x & -I_{xy} & 0 \\ mz_g & 0 & -mx_g & -I_{yx} & I_y & -I_{yz} \\ my_g & mx_g & 0 & -I_{zx} & -I_{zy} & I_z \end{bmatrix} \tag{8}$$

$$M_A=\begin{bmatrix} X_{\dot{u}} & X_{\dot{v}} & X_{\dot{w}} & X_{\dot{p}} & X_{\dot{q}} & X_{\dot{r}} \\ Y_{\dot{u}} & Y_{\dot{v}} & Y_{\dot{w}} & Y_{\dot{p}} & Y_{\dot{q}} & Y_{\dot{r}} \\ Z_{\dot{u}} & Z_{\dot{v}} & Z_{\dot{w}} & Z_{\dot{p}} & Z_{\dot{q}} & Z_{\dot{r}} \\ K_{\dot{u}} & K_{\dot{v}} & K_{\dot{w}} & K_{\dot{p}} & K_{\dot{q}} & K_{\dot{r}} \\ M_{\dot{u}} & M_{\dot{v}} & M_{\dot{w}} & M_{\dot{p}} & M_{\dot{q}} & M_{\dot{r}} \\ N_{\dot{u}} & N_{\dot{v}} & N_{\dot{w}} & N_{\dot{p}} & N_{\dot{q}} & N_{\dot{r}} \end{bmatrix} \tag{9}$$

上式中，m 表示过驱动 AUV 的质量；I_x，I_y，I_z表示过驱动 AUV 在机体坐标系中绕其坐标轴的转动惯量；x_g，y_g，z_g表示重心坐标在机体框架中的分量。M_A中的所有项均为一阶水动力导数，表征了 AUV 在流体环境中运动产生的附加质量效应。

$\mathrm{C(V)}$表示 AUV 的科里奥利-向心力矩矩阵，由向心力矩矩阵$\mathrm{C_A(V)}$和科里奥利力矩矩阵$\mathrm{C_{RB}(V)}$组成。向心力矩矩阵表示 AUV 旋转运动过程中由角速度引起的向心力矩的影响。科里奥利力矩矩阵$\mathrm{C_{RB}(V)}$描述了科里奥利力、科里奥利力矩与 AUV 的线速度和角速度之间的关系。

$$\mathrm{C(V)=C_A(V)+C_{RB}(V)} \tag{10}$$

$$C_A(V)=\begin{bmatrix} 0 & 0 & 0 & 0 & -a_3 & a_2 \\ 0 & 0 & 0 & a_3 & 0 & -a_1 \\ 0 & 0 & 0 & -a_2 & a_1 & 0 \\ 0 & -a_3 & a_2 & 0 & -b_3 & b_2 \\ a_3 & 0 & -a_1 & b_3 & 0 & -b_1 \\ -a_2 & a_1 & 0 & -b_2 & b_1 & 0 \end{bmatrix} \tag{11}$$

$$C_{RB}=\begin{bmatrix} 0 & 0 & 0 & m(y_gq+z_gr) & -m(x_gq-w) & -m(x_gr-v) \\ 0 & 0 & 0 & -m(y_gq+w) & m(z_gr+x_gp) & -m(y_gr-u) \\ 0 & 0 & 0 & -m(z_gp-v) & -m(z_gq+u) & m(x_gp+y_gq) \\ -m(y_gq+z_gr) & m(y_gq+w) & m(z_gp-v) & 0 & I_zr & I_yq \\ m(x_gq-w) & -m(z_gr+x_gp) & m(z_gq+u) & -I_zr & 0 & I_xp \\ m(x_gr-v) & m(y_gr-u) & -m(x_gp+y_gq) & -I_yq & -I_xp & 0 \end{bmatrix} \tag{12}$$

其中，a_1，a_2，a_3，b_1，b_2，b_3定义如下：

$$\begin{cases} a_1=X_{\dot{u}}u+X_{\dot{v}}v+X_{\dot{w}}w+X_{\dot{p}}p+X_{\dot{q}}q+X_{\dot{r}}r \\ a_2=X_{\dot{u}}u+Y_{\dot{v}}v+Y_{\dot{w}}w+Y_{\dot{p}}p+Y_{\dot{q}}q+Y_{\dot{r}}r \\ a_3=X_{\dot{u}}u+Y_{\dot{v}}v+Z_{\dot{w}}w+Z_{\dot{p}}p+Z_{\dot{q}}q+Z_{\dot{r}}r \\ b_1=X_{\dot{u}}u+Y_{\dot{v}}v+Z_{\dot{w}}w+K_{\dot{p}}p+K_{\dot{q}}q+K_{\dot{r}}r \\ b_2=X_{\dot{u}}u+Y_{\dot{v}}v+Z_{\dot{w}}w+K_{\dot{p}}p+M_{\dot{q}}q+M_{\dot{r}}r \\ b_3=X_{\dot{u}}u+Y_{\dot{v}}v+Z_{\dot{w}}w+K_{\dot{p}}p+M_{\dot{q}}q+N_{\dot{r}}r \end{cases} \tag{13}$$

$\mathrm{D(V)}$表示 AUV 在水下运动过程中受到的流体阻尼力和力矩。用于模拟 AUV 在流体中运动时，由于流体的粘性和阻力对 AUV 产生的阻尼效应，包括水摩擦、波浪阻尼等。

$$\mathrm{D(V)}=\begin{bmatrix} \mathrm{X_u+X_{u|u|}|u|} & 0 & 0 & 0 & 0 & 0 \\ 0 & \mathrm{Y_v+Y_{v|v|}|v|} & 0 & 0 & 0 & 0 \\ 0 & 0 & \mathrm{Z_w+Z_{w|w|}w} & 0 & 0 & 0 \\ 0 & 0 & 0 & \mathrm{K_p+K_{p|p|}|p|} & 0 & 0 \\ 0 & 0 & 0 & 0 & \mathrm{M_q+M_{q|q|}|q|} & 0 \\ 0 & 0 & 0 & 0 & 0 & \mathrm{N_r+N_{r|r|}|r|} \end{bmatrix} \tag{14}$$

在方程中，$\mathrm{X_u}$，$\mathrm{Y_v}$，$\mathrm{Z_w}$，$\mathrm{K_p}$，$\mathrm{M_q}$，$\mathrm{N_r}$表示 AUV 的一阶水动力系数，$\mathrm{X_{u|u|}}$，$\mathrm{Y_{v|v|}}$，$\mathrm{Z_{w|w|}}$，

$K_{p|p|}$，$M_{q|q|}$，$N_{r|r|}$表示 AUV 的二阶水动力系数，表明 AUV 在流体中运动时存在非线性水动力效应。这些系数由 AUV 的几何形状、流体的性质以及 AUV 的运动状态决定。

g (η)表示由重力和浮力作用产生的恢复力和力矩组成的矩阵。

$$g(\eta)=\begin{bmatrix}(W-B)\sin\theta\\-(W-B)\sin\varphi\cos\theta\\-(W-B)\cos\varphi\cos\theta\\-(y_gW-y_bB)\cos\varphi\cos\theta+(z_gW-x_bB)\sin\varphi\cos\theta\\(z_gW-z_bB)\sin\theta+(x_gW-x_bB)\cos\varphi\cos\theta\\-(x_gW-x_bB)\cos\varphi\cos\theta-(y_gW-y_bB)\sin\theta\end{bmatrix}\tag{15}$$

其中 B 表示 AUV 产生的浮力，W 表示作用在 AUV 上的引力。

τ表示由于 8 个推进器的共同作用而在 AUV 上产生的力和力矩：

$$\tau=[X,Y,Z,K,M,N]^T\tag{16}$$

τ_E表示 AUV 在作业过程中由于自身模型振动和外界干扰对其产生的扰动：

$$\tau_E=\left[d_u,d_v,d_w,d_p,d_q,d_r\right]^T\tag{17}$$

根据以上公式，在知道各个参数的情况下，根据 AUV 所受到的力和力矩，就可以求得加下来 AUV 的加速度和角加速度；根据加速度和角加速度就可以推算出接下来 AUV 速度、角速度。

3.3 AUV 推进系统建模与推力分配

3.3.1 推进系统模型

AUV 推进器分布见 2.1 结构部分，共有 8 个推进器，分别编号为 1-8。令 8 个推进器产生的推力为T_1-T_8，根据推进器的分布可以得出三个方向上产生的力 XYZ 和力矩 KMN 分别为：

$$\begin{cases}X=(T_1+T_2+T_3+T_4)\cos\alpha\\Y=(T_1-T_2-T_3+T_4)\sin\alpha\\Z=T_5+T_6+T_7+T_4\\K=f(-T_5+T_6-T_7+T_8)\\M=d(-T_5-T_6+T_7+T_8)\\N=c(T_1-T_2+T_3-T_4)\sin\alpha+b(T_5-T_6+T_7-T_8)\cos\alpha\end{cases}\tag{18}$$

根据以上公式，可以进一步推导出 AUV 所经历的推力和转矩矢量矩阵τ与 8 个推进器产生的推力和转矩之间的关系：

$$\tau=Bu_c\tag{19}$$

其中，$\tau=[X,Y,Z,K,M,N]$，$u_c=[T_1,T_2,T_3,T_4,T_5,T_6,T_7T_8]$，B 表示推力器的空间矢量排列矩阵。

$$B=\begin{bmatrix} \cos\alpha & \cos\alpha & \cos\alpha & \cos\alpha & 0 & 0 & 0 & 0 \\ \sin\alpha & -\sin\alpha & -\sin\alpha & \sin\alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & -f & f & -f & f \\ 0 & 0 & 0 & 0 & -d & -d & d & d \\ c\sin\alpha+b\cos\alpha & -c\sin\alpha-b\cos\alpha & c\sin\alpha+b\cos\alpha & -c\sin\alpha-b\cos\alpha & 0 & 0 & 0 & 0 \end{bmatrix} \tag{20}$$

3.3.2 推力分配方法

过驱动 AUV 上的推进器作为执行机构，推进器的数学模型为转速与推力的关系。八个推进器产生的推力直接作用于 AUV，引起位置和姿态的变化。直接逻辑分配法是一种接近最优能耗的推力分配方法，具有直观、简单、高效、实时性强等显著优点。本论文采用直接逻辑推力分配方法为每个推进器分配期望的转速；控制器负责输出每个自由度上的期望输入。直接逻辑推力分配方法将期望输入转化为八个推进器各自的期望转速。通过改变微控制器引脚上的 PWM 占空比，控制过驱动 AUV 上 8 个推进器的旋转方向和速度。最后，根据过驱动 AUV 上推进器的空间布置方式，将八个推进器产生的推力施加到 AUV 上，从而实现过驱动 AUV 的六自由度运动。

水平运动直接逻辑推力分配方法公式如下：

$$\begin{cases} T_{d1}(n)=X_c(n)+Y_c(n)+N_c(n) \\ T_{d2}(n)=X_c(n)-Y_c(n)+N_c(n) \\ T_{d3}(n)=X_c(n)-Y_c(n)+N_c(n) \\ T_{d4}(n)=X_c(n)+Y_c(n)-N_c(n) \end{cases} \tag{21}$$

水平面内同向旋转的 4 个推进器均可实现过驱动 AUV 的前进和后退运动。推进器 1 和推进器 4 同向旋转，推进器 2 和推进器 3 反向旋转，实现过驱动 AUV 的向右运动。同样地，通过反转四个水平推进器的旋转方向，过驱动 AUV 可以向左移动。保持推进器 1 和推进器 2 的速度不变，改变推进器 3 和推进器 4 的旋转方向，过驱动 AUV 可以进行左右转弯。其中$X_c(n)$、$Y_c(n)$和$N_c(n)$是控制器产生的与水平运动相关的期望输入，$T_{d1}(n)$、$T_{d2}(n)$、$T_{d3}(n)$和$T_{d4}(n)$是四个推进器在水平面内的期望转速，分别编号为 1、2、3 和 4。

垂直运动直接逻辑推力分配方法公式如下：

$$\begin{cases} T_{d5}(n)=Z_c(n)-K_c(n)-M_c(n) \\ T_{d6}(n)=Z_c(n)+K_c(n)-M_c(n) \\ T_{d7}(n)=Z_c(n)-K_c(n)+M_c(n) \\ T_{d8}(n)=Z_c(n)+K_c(n)+M_c(n) \end{cases} \tag{22}$$

垂直面内所有四个推进器同向旋转均可实现过驱动 AUV 的下潜和上浮运动。推进器 5 和 7 反向旋转，推进器 6 和 8 同向旋转，实现过驱动 AUV 的逆时针翻

滚运动。同样，通过反转四个垂直推进器的旋转方向，超驱动 AUV 可以执行顺时针滚动。推进器 5 和 6 沿相反方向旋转，而推进器 7 和 8 沿同一方向旋转，以实现过度驱动的 AUV 的向下俯仰运动。同样，通过反转四个垂直推进器的旋转方向，超驱动的 AUV 可以执行向上的俯仰。其中 $Z_c(n)$、$K_c(n)$和$M_c(n)$是与控制器产生的垂直运动相关的预期转速输入，$T_{d5}(n)$、$T_{d6}(n)$、$T_{d7}(n)$和$T_{d8}(n)$是垂直平面中四个推进器的预期转速，分别编号为 5、6、7 和 8。

4 控制器设计

该论文实现的控制策略为双环控制，外环为模型预测控制（MPC），内环为变论域 S 平面算法（S - VUD FLC），再加上执行机构的推力分配控制器，实现了 AUV 的轨迹跟踪。

4.1 外环 MPC 运动学控制器设计

外环位置控制器的作用是根据系统状态和目标函数在每个控制周期实时计算最优控制输入，调整控制序列以最大程度优化跟踪性能，使 AUV 准确跟踪期望轨迹。考虑 AUV 实际控制输入约束（如线速度和角速度变化范围），引入控制增量以减少控制模型误差和不确定外部干扰对实际 AUV 模型参数的影响。

该部分控制器主要分为 3 个部分，分别是运动学方程线性化处理，系统未来状态预测以及系统预测模型的约束条件与优化。

4.1.1 运动学方程线性化处理

基于实际的实验过程以及 AUV 自身的参数，横滚角φ，和横滚角速度 p 通常都很小，即使受到干扰也能很快自稳，所以可以被简化，以减少运算量。

外环控制器的输入为期望位置姿态和实际位置姿态，以及当前的速度/角速度，而且由于控制是实时的，在任意时刻 t，有：

实际位置:

$$\eta_d(t) = [x_d(t), y_d(t), z_d(t), \theta_d(t), \psi_d(t)] \tag{23}$$

期望位置：

$$\eta(t) = [x(t), y(t), z(t), \theta(t), \psi(t)] \tag{24}$$

当前速度/角速度:

$$V_d(t) = [u_d(t), v_d(t), w_d(t), q_d(t), r_d(t)] \tag{25}$$

运动学方程：

$$\dot{\eta}_d(t) = J(\eta_d)V_d(t) \tag{26}$$

对以上方程在(η_d, V_d)处进行线性化：

$$\dot{\tilde{\eta}} = \begin{bmatrix} \dot{x}(t) - \dot{x}_d(t) \\ \dot{y}(t) - \dot{y}_d(t) \\ \dot{z}(t) - \dot{z}_d(t) \\ \dot{\theta}(t) - \dot{\theta}_d(t) \\ \dot{\psi}(t) - \dot{\psi}_d(t) \end{bmatrix} = a\tilde{\eta} + b\tilde{v} = \begin{bmatrix} 0 & 0 & 0 & a_{14} & a_{15} \\ 0 & 0 & 0 & a_{24} & a_{25} \\ 0 & 0 & 0 & a_{34} & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x(t) - x_d(t) \\ y(t) - y_d(t) \\ z(t) - z_d(t) \\ \theta(t) - \theta_d(t) \\ \psi(t) - \psi_d(t) \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} & b_{13} & 0 & 0 \\ b_{21} & b_{22} & b_{23} & 0 & 0 \\ b_{31} & 0 & b_{33} & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & b_{55} \end{bmatrix} \begin{bmatrix} u(t) - u_d(t) \\ v(t) - v_d(t) \\ w(t) - w_d(t) \\ q(t) - q_d(t) \\ r(t) - r_d(t) \end{bmatrix} \tag{27}$$

其中，$a_{14}, a_{15}, a_{24}, a_{25}, a_{34}, b_{11}, b_{12}, b_{13}, b_{21}, b_{22}, b_{23}, b_{31}, b_{33}, b_{55}$参数如下：

$$\begin{cases} a_{14} = -u_d \sin\theta_d \cos\psi_d + w_d \cos\theta_d \cos\psi_d \\ a_{24} = -u_d \sin\theta_d \sin\psi_d - w_d \sin\theta_d \sin\psi_d \\ a_{34} = -u_d \cos\theta_d - w_d \sin\theta_d \\ a_{15} = -u_d \cos\theta_d \cos\psi_d - v_d \cos\psi_d - w_d \sin\theta_d \sin\psi_d \\ a_{25} = u_d \cos\theta_d \cos\psi_d - v_d \sin\psi_d + w_d \sin\theta_d \cos\psi_d \\ b_{11} = \cos\theta_d \cos\psi_d \\ b_{12} = -\sin\psi_d \\ b_{13} = \sin\theta_d \cos\psi_d \\ b_{21} = \cos\theta_d \sin\psi_d \\ b_{22} = \cos\psi_d \\ b_{23} = \sin\theta_d \sin\psi_d \\ b_{31} = -\sin\theta_d \\ b_{33} = \cos\theta_d \\ b_{55} = \dfrac{1}{\cos\theta_d} \end{cases} \tag{28}$$

进行 Euler 正向方程来离散方程，可得：

$$\tilde{\eta}(k+1) = a_l\tilde{\eta}(k) + b_l\tilde{V}(k) \tag{29}$$

其中$\mathrm{a_l} = \mathrm{I_5} + \mathrm{aT}$，$\mathrm{b_l} = \mathrm{bT}$，$\mathrm{I_5}$是一个 5*5 的单位矩阵，T 表示采样时间。

为了提高稳定性并消除稳态下的静态误差，考虑了 AUV 在水下工作时出现的建模误差和过程噪声。在基于状态空间方程的预测控制中，采用基于增强状态的状态空间模型，引入控制输入 $\mathrm{u(k)} = \tilde{\mathrm{V}}(\mathrm{k}) - \tilde{\mathrm{V}}(\mathrm{k}-1)$的增量，并将受控系统的状态变量重置为 $\mathrm{x(k)} = [\tilde{\eta}(k), \tilde{\mathrm{V}}(\mathrm{k}-1)]$，以上方程变为：

$$\mathrm{x(k+1)} = \begin{bmatrix} \tilde{\eta}(k+1) \\ \tilde{V}(k) \end{bmatrix} = \begin{bmatrix} \mathrm{a_l} & \mathrm{b_l} \\ 0_{5\times5} & \mathrm{I_5} \end{bmatrix} \begin{bmatrix} \tilde{\eta}(k) \\ \tilde{V}(k-1) \end{bmatrix} + \begin{bmatrix} \mathrm{b_l} \\ \mathrm{I_5} \end{bmatrix} [\tilde{V}(k) - \tilde{V}(k-1)] = \mathrm{Ax(x)} + \mathrm{Bu(u)} \tag{30}$$

式中，$\mathrm{A} = \begin{bmatrix} \mathrm{a_l} & \mathrm{b_l} \\ 0_{5\times5} & \mathrm{I_5} \end{bmatrix}$，$\mathrm{B} = \begin{bmatrix} \mathrm{b_l} \\ \mathrm{I_5} \end{bmatrix}$。令 $\mathrm{y(k)} = \mathrm{Cx(k)}$，$\mathrm{C} = [\mathrm{I_5}, 0_{5\times5}]$

结合以上公式，可得：

$$\begin{cases} \mathrm{x(k+1)} = \mathrm{Ax(x)} + \mathrm{Bu(u)} \\ \mathrm{y(k)} = \mathrm{Cx(k)} \end{cases} \tag{31}$$

4.1.2 系统的未来状态预测

假设预测范围为N_p，控制范围为 N_c（$N_p \geq N_c$）。 通过对上述方程的多步

推导，系统在N_p时域内的状态预测值为：

$$
\begin{aligned}
&x(k+1)=Ax(k)+Bu(k)\\
&x(k+2)=Ax(k+1)+Bu(k+1)=A^2x(k)+ABu(k)+Bu(k+1)\\
&x(k+3)=Ax(k+2)+Bu(k+2)=A^3x(k)+A^2Bu(k)+ABu(k+1)+Bu(k+2)\\
&\qquad\vdots\\
&x(k+N_c)=Ax(k+N_c)+Bu(k+N_c)=A^{N_c}x(k)+A^{N_c-1}Bu(k)+A^{N_c-2}Bu(k+1)+\cdots+A^0Bu(k+N_c-1)\\
&\qquad\vdots\\
&x(k+N_p)=A^{N_p}x(k)+A^{N_p-1}Bu(k)+A^{N_p-2}Bu(k+1)+\cdots+A^0Bu(k+N_p-1)
\end{aligned}
\tag{32}
$$

同样的，经过多次推到，可以得到系统的 Np 时域内设置的输出方程：

$$
\begin{aligned}
&y(k+1)=Cx(k+1)=Ax(k)+CBu(k)\\
&y(k+2)=CAx(k+1)+CBu(k+1)=CA^2x(k)+CABu(k)+CBu(k+1)\\
&y(k+3)=CAx(k+2)+CBu(k+2)=CA^3x(k)+CA^2Bu(k)+CABu(k+1)+CBu(k+2)\\
&\qquad\vdots\\
&y(k+N_c)=CAx(k+N_c)+CBu(k+N_c)=CA^{N_c}x(k)+CA^{N_c-1}Bu(k)+CA^{N_c-2}Bu(k+1)+\ldots+CA^0Bu(k+N_c-1)\\
&\qquad\vdots\\
&y(k+N_p)=CA^{N_p}x(k)+CA^{N_p-1}Bu(k)+CA^{N_p-2}Bu(k+1)+\ldots+CA^0Bu(k+N_p-1)
\end{aligned}
\tag{33}
$$

对以上公式进行简化，令：

$$
\Upsilon=\begin{bmatrix} y(k+1)\\ y(k+2)\\ \ldots\\ y(k+N_c)\\ \ldots\\ y(k+N_p)\end{bmatrix},\ \Psi=\begin{bmatrix} \mathrm{CA}\\ CA^2\\ \ldots\\ CA^{N_c}\\ \ldots\\ CA^{N_p}\end{bmatrix},\ \mathrm{U}=\begin{bmatrix} \mathrm{u(k)}\\ \mathrm{u(k+1)}\\ \mathrm{u(k+2)}\\ \ldots\\ u(k+N_c-1)\end{bmatrix}
\tag{34}
$$

$$
\Theta=\begin{bmatrix}
\mathrm{CB} & 0 & 0 & \ldots & 0\\
\mathrm{CAB} & \mathrm{CB} & 0 & \ldots & 0\\
\ldots & \ldots & \ldots & \ddots & \ldots\\
CA^{N_c-1}B & CA^{N_c-1}B & CA^{N_c-3}B & \ldots & CA^0B\\
\ldots & \ldots & \ldots & \ddots & \ldots\\
CA^{N_p-1}B & CA^{N_p-2}B & CA^{N_p-3}B & \ldots & CA^{N_p-N_c}B
\end{bmatrix}
\tag{35}
$$

于是，输出方程可以写成更紧凑的形式：

$$
\Upsilon=\Psi\mathrm{x(k)}+\Theta\mathrm{U}
\tag{36}
$$

4.1.3 AUV 系统预测控制器的约束优化

由于是已经建立的误差方程，所以控制目标是使参考轨迹的跟踪误差逐渐收敛。因此，系统输出的参考值可以定义为:

$$
\Upsilon_r=[y_r(k+1)\ \ y_r(k+1)\ \ \ldots\ \ y_r(k+1)\ \ \ldots\ \ y_r(k+1)]^T=[0\ \ 0\ \ \ldots\ \ 0\ \ \ldots\ \ 0]^T
\tag{37}
$$

优化目标函数可以定义为：

$$
\min \mathrm{J(k)}=\sum_{\mathrm{i}=1}^{N_p}||\mathrm{y(k+i|k)}-\mathrm{y_r(k+i)}||_{\mathrm{Q}}^2+\sum_{\mathrm{i}=0}^{N_c}||\mathrm{u(k+i|k)}||_{\mathrm{R}}^2
\tag{38}
$$

其中 Q 和 R 分别表示输出信号和控制信号的加权矩阵。

令 $\mathrm{E}=\Psi\mathrm{x(k)}$，$\mathrm{Q_Q}=\mathrm{I}_{N_p}\otimes\mathrm{Q}$，$\mathrm{R_R}=\mathrm{I}_{N_c}\otimes\mathrm{R}$，$\tilde{\Upsilon}=(\Upsilon-\Upsilon_\mathrm{r})$。其中$\otimes$代表克罗内克积。对上式进行化简：

$$
\begin{aligned}
&\min \mathrm{J(k)} = \tilde{\Upsilon}^{\mathrm{T}}\mathrm{Q_Q}\tilde{\Upsilon} + \mathrm{U^T R_R U} \\
&= (\Upsilon - \Upsilon_{\mathrm{r}})^{\mathrm{T}}\mathrm{Q_Q}(\Upsilon - \Upsilon_{\mathrm{r}}) + \mathrm{U^T R_R U} \\
&= (\Psi \mathrm{x(k)} + \Theta \mathrm{U} - \Upsilon_{\mathrm{r}})^{\mathrm{T}}\mathrm{Q_Q}(\Psi \mathrm{x(k)} + \Theta \mathrm{U} - \Upsilon_{\mathrm{r}}) + \mathrm{U^T R_R U} \\
&= \mathrm{U^T}(\Theta^{\mathrm{T}}\mathrm{Q_Q}\Theta + \mathrm{R_R})\mathrm{U} + 2\mathrm{E^T Q_Q}\Theta\mathrm{U} + \mathrm{E^T Q_Q E} - \Upsilon_{\mathrm{r}}\mathrm{Q_Q}\Theta\mathrm{U} + \Upsilon_{\mathrm{r}}^{\mathrm{T}}\mathrm{Q_Q}\Upsilon - 2\Upsilon_{\mathrm{r}}^{\mathrm{T}}\mathrm{Q_Q E}
\end{aligned}
\tag{39}
$$

由于$\mathrm{E^T Q_Q E} - \Upsilon_{\mathrm{r}}\mathrm{Q_Q}\Theta\mathrm{U} + \Upsilon_{\mathrm{r}}^{\mathrm{T}}\mathrm{Q_Q}\Upsilon - 2\Upsilon_{\mathrm{r}}^{\mathrm{T}}\mathrm{Q_Q E}$是一个常数，在求解目标函数时可以省略。并且令$\mathrm{H} = \Theta^{\mathrm{T}}\mathrm{Q_Q}\Theta$，$\mathrm{g} = \mathrm{E^T Q_Q}\Theta$，于是目标函数就可以被重新写成：

$$
\min_{\mathrm{U}} \mathrm{J} = 2(\frac{1}{2}\mathrm{U^T HU} + \mathrm{g^T U}) \Leftrightarrow \min_{\mathrm{U}} \mathrm{J} = \frac{1}{2}\mathrm{U^T HU} + \mathrm{g^T U} \tag{40}
$$

对于控制量和控制增量，存在以下递推公式：

$$
\begin{aligned}
&\tilde{V}(k) = \tilde{V}(k-1) + u(k) \\
&\tilde{V}(k+1) = \tilde{V}(k) + u(k+1) = \tilde{V}(k-1) + u(k) + u(k+1) \\
&\qquad\qquad\qquad\qquad\qquad\qquad \vdots \\
&\tilde{V}(k+N_c-1) = \tilde{V}(k+N_c-2) + u(k+N_c-1) = \tilde{V}(k-1) + u(k) + u(k+1) + \ldots + u(k+N_c-1)
\end{aligned}
\tag{41}
$$

以上公式可写为：

$$
V_A = \begin{bmatrix} \tilde{\mathrm{V}}(\mathrm{k}) \\ \tilde{\mathrm{V}}(\mathrm{k}+1) \\ \tilde{\mathrm{V}}(\mathrm{k}+2) \\ \ldots \\ \tilde{\mathrm{V}}(\mathrm{k}+\mathrm{N_c}-1) \end{bmatrix} = \begin{bmatrix} \tilde{\mathrm{V}}(\mathrm{k}-1) \\ \tilde{\mathrm{V}}(\mathrm{k}-1) \\ \tilde{\mathrm{V}}(\mathrm{k}-1) \\ \ldots \\ \tilde{\mathrm{V}}(\mathrm{k}-1) \end{bmatrix} + \begin{bmatrix} \mathrm{I_2} & 0 & 0 & \ldots & 0 \\ \mathrm{I_2} & \mathrm{I_2} & 0 & \ldots & 0 \\ \mathrm{I_2} & \mathrm{I_2} & \mathrm{I_2} & \ldots & 0 \\ \ldots & \ldots & \ldots & \ddots & 0 \\ \mathrm{I_2} & \mathrm{I_2} & \mathrm{I_2} & \ldots & \mathrm{I_2} \end{bmatrix} \begin{bmatrix} \mathrm{u(k)} \\ \mathrm{u(k+1)} \\ \mathrm{u(k+2)} \\ \ldots \\ \mathrm{u(k+N_c-1)} \end{bmatrix} = \mathrm{V_T} + \mathrm{A_I}U \tag{42}
$$

由于推进器的功率限制和水下拖曳环境的影响，AUV 的速度和速度增量是有界的。设状态变量中控制输入的幅值有一个上界V_{max}和一个下界V_{min}，控制输入增量$u(k)$的幅值有一个上界u_{max}和一个下界u_{min}。

$$
\begin{aligned}
u_{min} \le u(k) \le u_{max} \\
V_{min} \le V_A \le V_{max}
\end{aligned}
\tag{43}
$$

控制输入增量 U 的幅值也是有限的，分别设为上界U_{max}和下界U_{min}。因此，可以得到以下控制输入的不等式约束：

$$
V_{min} \le V_T + A_I U \le V_{max} \rightarrow \begin{cases} A_I U \le V_{max} - V_T \\ -A_I U \le -V_{min} + V_T \end{cases} \tag{44}
$$

$$
U_{min} \le U \le U_{max} \tag{45}
$$

综上所述，AUV 的预测三维轨迹跟踪控制问题已转化为标准二次规划(QP)问题：

$$
\begin{aligned}
U^*(k) = \min_U J = \frac{1}{2}U^T HU + g^T U \\
s.t. \begin{cases} A_I U \le V_{max} - V_T \\ -A_I U \le -V_{min} + V_T \\ U_{min} \le U \le U_{max} \end{cases}
\end{aligned}
\tag{46}
$$

通过求解 QP 问题，可以得到每个时刻N_c域内的最优控制作用$U^*(k)$。将向量

$U^*(k)$中的第一个元素$u^*(k)$作为 k 时刻的最优控制作用。由于$u^*(k)$是最优控制增量输入，需要经过如下公式得到当前采样时刻 k 处的线速度和角速度的最优参考值$V_r{}^*(k)$。

$$V_r{}^*(k) = \widetilde{\mathrm{V}}(\mathrm{k}-1) + u^*(k) + V_d(k) \tag{47}$$

$$V_r{}^*(k) = \begin{bmatrix} u_r \\ v_r \\ w_r \\ p_r \\ r_r \end{bmatrix}_k \tag{48}$$

至此，外环运动学控制器的设计已经完成。将$V_r{}^*(k)$作为动态控制器的参考值。接下来，开始内环动态控制器的设计。

4.2 内环变论域 S 平面动力学控制器设计

给部分控制器，论文对传统的模糊控制器加以改进，结合了 PID 算法的积分思想，提出了全新的 P - S - VUD FLC 算法。

4.2.1 常规变论域模糊控制器

常规的变论域模糊控制器的近似表达式可以描述为

$$\mathrm{y} = \mathrm{f}(\mathrm{x}/\mathrm{s}(\mathrm{x})) \tag{49}$$

其中 s (x)表示比例因子，x 表示控制器的输入，y 表示控制器的输出。本文直接采用已经确定的比例因子，主要有两种常用的比例因子形式.

形式一：$\mathrm{s}(\mathrm{x}) = (\frac{|\mathrm{x}|}{\mathrm{E}})^{\tau} + \varepsilon$，$\tau > 0$。ε是一个非常小的正数。

形式二：$\mathrm{s}(\mathrm{x}) = 1 - \lambda \mathrm{e}^{-\mathrm{k}x^2}$，$\lambda \in (0,1)$，$\mathrm{k} > 0$

S 面控制策略基于模糊控制思想，用" S "形函数面代替传统的模糊控制器。它使用 sigmoid 函数来构造 S 面控制器函数，使得控制器更容易设计。经典的具有非线性控制面的 S 面控制更适用于自主式水下机器人的运动控制。

4.2.2 改进思路

变论域模糊控制器分为两个部分：比例因子部分和模糊逻辑部分。对于模糊逻辑部分，由于 S 面控制策略是基于模糊控制理论设计的，本论文尝试将两者结合，在保留比例因子部分的同时使用 S 面(它可以被称为基函数)作为替代，从而简化了变论域模糊控制器。这样就形成了一种新的算法，即 S - VUD FLC 算法。它是变论域模糊控制器和 S 面算法的结合，结合了变论域模糊控制器的强鲁棒性和 S 面算法的易构造性。

以下是新算法的构造步骤：

步骤一：构造 sigmoid 函数；

$$\mathrm{tansig(x)} = \frac{\mathrm{e^x} - \mathrm{e^{-x}}}{\mathrm{e^x} + \mathrm{e^{-x}}} \tag{50}$$

步骤二：构造 S -平面，设x_1和x_2为输入，y 为输出，从而得到：

$$\mathrm{y} = \beta_1 \mathrm{tansig}(\alpha_1 x_1) + \beta_2 \mathrm{tansig}(\alpha_2 x_2) \tag{51}$$

步骤三：S 面控制器用$y_s = f_s(X)$表示，其中 X 表示输入向量。S-VUD FLC 控制器可以表示为：

$$y_s^{'} = f_s(\frac{x_1}{\alpha(x_1)}, \frac{x_2}{\alpha(x_2)})/\beta(pX), p = [0.5,0.5] \tag{52}$$

$$\alpha(\mathrm{x}) = 1 - \gamma \mathrm{e}^{-0.5x^2}(\gamma < 1), \beta(pX)\text{确保了}y_s^{'}\text{足够大} \tag{53}$$

$$f_s(X) = \lambda tansig(cx_1) + (1 - \lambda) tansig(cx_2) \tag{54}$$

在上述公式中，α (x)和β (pX)分别表示输入端和输出端的比例因子(如果取定值,它可以退化为增益)，p 为权向量。因此，因此，S-VUD FLC 算法可以表示为一个 S 平面函数和缩放因子的组合。因子 c 会引起水平轴伸缩，改变输入的有效范围。因子λ可以改变输入的权重；当输入噪声显著时，需要增加误差的权重来降低噪声影响。

比例因子的作用也会影响控制性能。γ越接近 1，控制器的变化越陡峭，稳态误差越小，响应越快，但振荡越强；γ越小，控制器变化越平缓，响应速度越慢，振荡越小。

为减小系统的稳态误差，增强系统在不确定水下环境中的抗扰动能力，提高系统的稳定性，借鉴 PID 控制器的设计思想，对 S - VUD FLC 进行改进。通过在 S - VUD FLC 的表达式中引入积分项，论文将新的控制器命名为改进的变论域 S 面控制器(P-S-VUD FLC)。其表达式如下：

$$y_s^{'} = f_s(\frac{x_1}{\alpha(x_1)}, \frac{x_2}{\alpha(x_2)})/\beta(pX) + K_i \int_0^t x_2 dt, p = [0.5,0.5] \tag{55}$$

$$\alpha(\mathrm{x}) = 1 - \gamma \mathrm{e}^{-0.5x^2}(\gamma < 1), \beta(pX)\text{确保了}y_s^{'}\text{足够大} \tag{56}$$

$$f_s(X) = \lambda tansig(cx_1) + (1 - \lambda) tansig(cx_2) \tag{57}$$

在上式中，K_i是积分项的系数。新的控制器结合了变论域模糊控制器的强鲁棒性和 S 面算法易于构造的特点。其优点是非常明显的：它大大简化了变论域模

糊控制器的设计，使得控制器的设计更加灵活，更易于实现。模型仿真非常方便，系统的性能分析和优化更加直观，加快了控制算法的开发和验证过程。

4.2.3 针对过驱动 AUV 的 P - S - VUD FLC 设计过程

步骤一：令外环运动学控制器产生的期望速度$V_r{}^*(k)$与传感器感知的实际速度$V(k)$之差记为e_V，的导数记为$\dot{e_V}$。

$$e_V = V_r{}^*(k) - V(k) = \begin{bmatrix} u_r - u \\ v_r - v \\ w_r - w \\ q_r - q \\ r_r - r \end{bmatrix}_k \tag{58}$$

步骤二：速度偏差e_V及其导数$\dot{e_V}$作为内环动态控制器的输入。如果我们令$p_r = 0$，则e_V可以写成如下形式：

$$e_V = \begin{bmatrix} u_r - u \\ v_r - v \\ w_r - w \\ q_r - q \\ r_r - r \end{bmatrix}_k \tag{59}$$

步骤三：对 AUV 的每个自由度设计 P - S - VUD FLC，这 6 个改进的 Variable Universe S - plane 控制器共同组成内环动态控制器，输出推力分配控制器的期望输入，如下图所示：

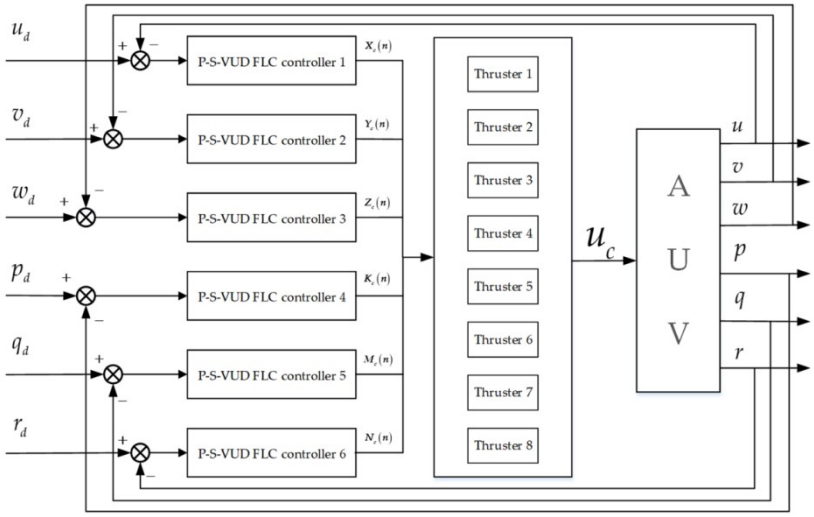

图 4 内环控制器与推挤分配控制器示意图

4.3 推力分配控制器

将内环控制器的输出$[X_c(n), Y_c(n), Z_c(n), K_c(n), M_c(n), N_c(n)]^T$作为推力分配控制器的输入，由章节 3.2 可知，采用直接逻辑推力分配，有：

$$\begin{cases} T_{d1}(n) = X_c(n) + Y_c(n) + N_c(n) \\ T_{d2}(n) = X_c(n) - Y_c(n) + N_c(n) \\ T_{d3}(n) = X_c(n) - Y_c(n) + N_c(n) \\ T_{d4}(n) = X_c(n) + Y_c(n) - N_c(n) \\ T_{d5}(n) = Z_c(n) - K_c(n) - M_c(n) \\ T_{d6}(n) = Z_c(n) + K_c(n) - M_c(n) \\ T_{d7}(n) = Z_c(n) - K_c(n) + M_c(n) \\ T_{d8}(n) = Z_c(n) + K_c(n) + M_c(n) \end{cases} \tag{60}$$

$T_{d1}(n)$~$T_{d8}(n)$就是 8 个推进器产生的推力，再根据公式

$$\tau = \mathrm{B}u_c \tag{61}$$

$$\tau = [\mathrm{X}, \mathrm{Y}, \mathrm{Z}, \mathrm{K}, \mathrm{M}, \mathrm{N}] \tag{61}$$

$$u_c = [T_{d1}(n), T_{d2}(n), T_{d3}(n), T_{d4}(n), T_{d5}(n), T_{d6}(n), T_{d7}(n), T_{d8}(n)] \tag{62}$$

$$\mathrm{B} = \begin{bmatrix} \cos\alpha & \cos\alpha & \cos\alpha & \cos\alpha & 0 & 0 & 0 & 0 \\ \sin\alpha & -\sin\alpha & -\sin\alpha & \sin\alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & -f & f & -f & f \\ 0 & 0 & 0 & 0 & -d & -d & d & d \\ c\sin\alpha + b\cos\alpha & -c\sin\alpha - b\cos\alpha & c\sin\alpha + b\cos\alpha & -c\sin\alpha - b\cos\alpha & 0 & 0 & 0 & 0 \end{bmatrix} \tag{63}$$

即可求得由 8 个推进器所产生的合力，即作用在 AUV 上的推力和扭矩。

5 程序仿真

仿真程序见超链接：代码

5.1 参数设置

根据论文所示，以下为现有的参数：

表 2 仿真参数

参数	取值
m	177kg
B	1158kN
M_{RB}	diag{117,117,117,10.1,11.8,13.4}
M_A	diag{58.4,23.8,23.8,3.38,1.18,2.67}
D(v)	diag{120+90|u|,90+90|v|,150+120|w|,15+10|p|,15+12|q|,18+15|r|}

本文将模拟的海流扰动模型τ_E加入到 AUV 线速度矢量 $V_1 = [\,u\,, v\,, w]^T$ 的各个分量中，如式所示:

$$\tau_E = \left[0.5\sin\left(\frac{t}{75}\right)\cos\left(\frac{t}{150}\right)\right] \tag{64}$$

为了验证所设计控制器对曲线轨迹跟踪的控制性能，将曲线参考轨迹定义为

如下，AUV 的初始状态设置为[0,0,0,0,0]^{T}：

$$\Upsilon_d(t) = \begin{cases} 0.5\sin(0.8t) \\ 0.5 - 0.5\cos(0.8t) \\ 0.3t \end{cases} \tag{65}$$

内环控制器和外环控制器参数如下：

表 3 控制器仿真参数

参数	取值
T（采样时间）	0.0005
Q_i	100
R_i	0.1
N_P	20
N_c	10
V_{max}	[0.7, 0.4, 0.7, 1.25, 1.25]
V_{min}	[-0.7, -0.4, -0.7, -1.25, -1.25]
U_{max}	[0.08, 0.08, 0.08, 0.15, 0.15]
U_{min}	[-0.08, -0.8, -0.08, -0.15, -0.15]
γ	0.85
λ	0.5
c	0.1
K_i	100
p	[0.5, 0.5]

5.2 仿真结果

5.2.1 无扰动下的仿真结果

在无扰动时候，xyz 的时间曲线和位置、姿态误差曲线如下所示：

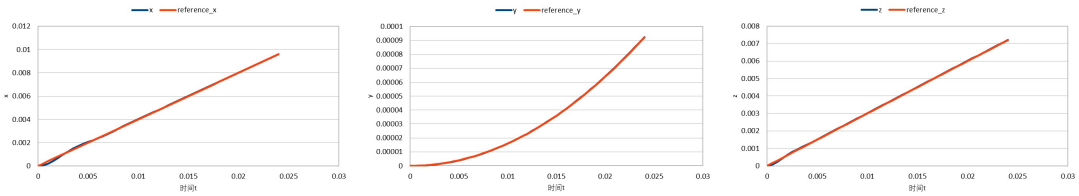

图 5 0.03 秒内实际值与参考值对比

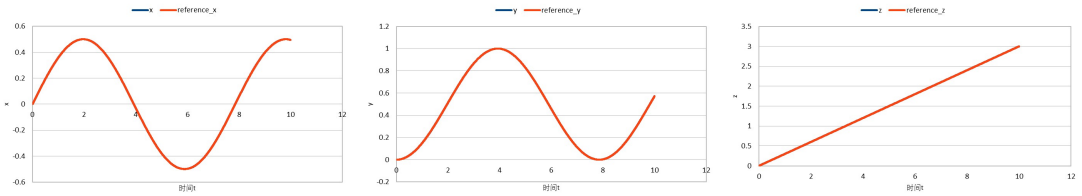

图 6 10 秒内实际值与参考值对比

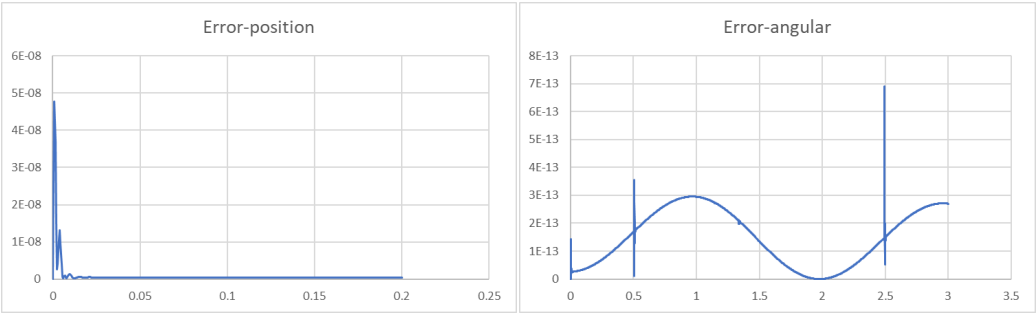

图 7 位置误差与姿态误差

由上图可见，在无扰动情况该控制方法具有极好的精确性、准确性和快速性，仅在前 0.05 秒有些许误差，在 0.05 秒之后即可达到位置几乎零误差。

5.2.2 扰动下的仿真结果

对 AUV 添加扰动，其扰动方程为：$\tau_E = \left[0.5\sin\left(\frac{t}{75}\right)\cos\left(\frac{t}{150}\right)\right]$。仿真结果如下：

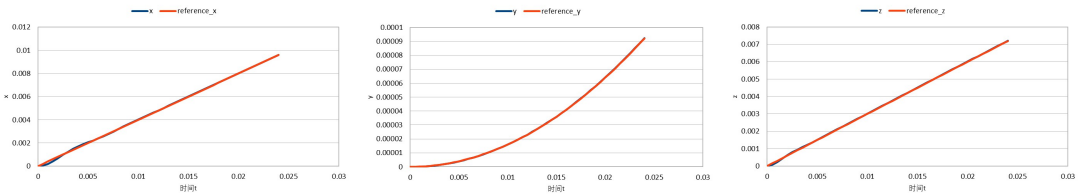

图 8 0.03 秒内实际值与参考值对比

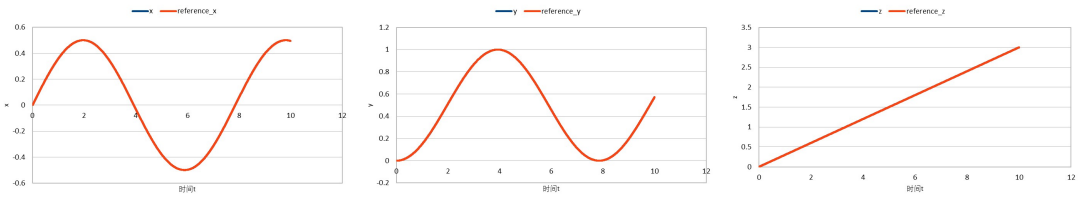

图 9 10 秒内参考值与实际值对比

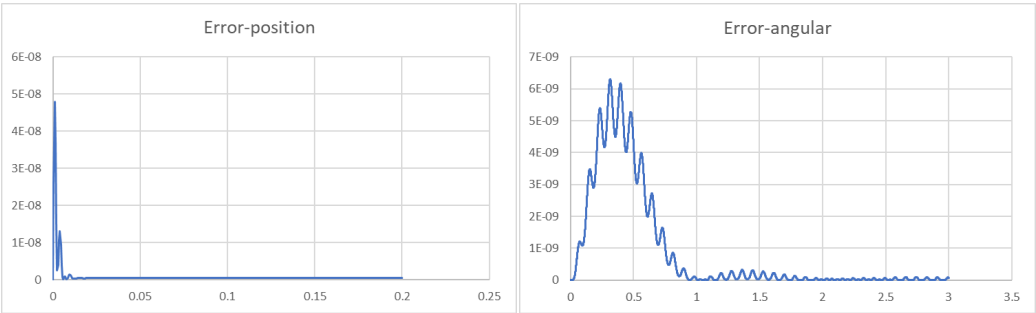

图 10 位置误差与姿态误差

由上图可知即使添加了扰动，系统任然具有较好的精确性、准确性和快速性，同样在 0.05 秒以内就达到了位置上几乎 0 误差。

下图为仿真程序的可视化结果：

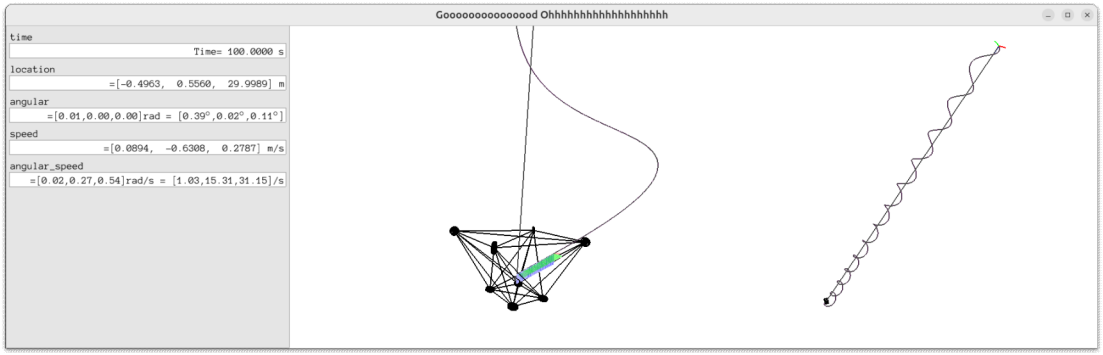

图 11 程序运行可视化窗口

从运行窗口可以直观的看到每个时刻 AUV 的位置，姿态，速度和角速度，以及 AUV 运行时的跟随视角和全景视角。

6 程序说明

程序分为 4 个模块，分别是控制器模块，模型模块，参数模块和可视化模块。

6.1 main 函数

在 main 函数里实例化 PARAMS 类，并调用计算函数，传入参数 0 或 1,0 代表计算完设定的目标时间后再可视化，1 代表一边计算一边可视化。

```
PARAMS params;
params.Calculate(0);    // 0 计算完在绘制  1 边计算边绘制
```

图 12 main 函数主要代码

6.2 参数模块

参数模块包含了 AUV 仿真过程中的实时变化的参数，包括时间，速度、角速度、位置、姿态、推力、力矩等变量，且由于其中的变量是存储在 PARAMS 内里，所以可以在各个模块当中传递。本模块中通过调用其余模块，以计算 AUV 的实时数据变化，并将数据传递给可视化模块进行可视化。

时间，速度、角速度、位置、姿态、推力、力矩等变量在 PARAMS 内中提前设置好，在后续各种变量发生变化之后均隶属于 PARAMS 类，此处就不再赘述。

下图为 PARAMS 类的计算核心代码，接收 main 函数所调用的参数，当参数为 0 时，开始模拟 AUV 在时间 t 内的路径跟踪，并在计算完成后写入数据到本地保存，并进行可视化：

```
MPC mpc;
void PARAMS::Calculate(int mode){
    vector<string>filenames={"../B_Times.txt","../B_Eta_Tau_E.txt","../B_Reference_Eta_Tau_E.txt","../
    B_Tau_Tau_E.txt"};
    if(mode==0){
        for(t=0;t<=t_max;t+=T){
            Update_Params();
            Tau=Controller_trace_to_Force(t,T,t_max,Eta,V,Reference_Eta);
            V_dot=dynamics(M_,C_V,D_V,g_Eta,Tau,tau_E[int(t/T)],V);
            V+=V_dot*T;
            Eta_dot=kinematics(J_eta,V);
            Eta+=Eta_dot*T;
        }
        Write_data_v(Times,filenames[0]);
        Write_data_vv(Eta_list,filenames[1]);
        Write_data_vv(Reference_Eta,filenames[2]);
        Write_data_vv(Tau_list,filenames[3]);
        Visual.Init();
        while(true)
            Visual.Draw(mode,t,T,position_list,pose_list,speed_list,angular_speed_list,poses);
    }
```

图 20 PARAMS 类核心函数 1

当调用传参为 1 时，即在 for 循环中一边计算一边一边可视化，当计算结束后，写入数据到本地，并可视化：

```
    if(mode==1){
        Visual.Init();
        for(t=0;t<=t_max;t+=T){
            Update_Params();
            Tau=Controller_trace_to_Force(t,T,t_max,Eta,V,Reference_Eta);
            V_dot=dynamics(M_,C_V,D_V,g_Eta,Tau,tau_E[int(t/T)],V);
            V+=V_dot*T;
            Eta_dot=kinematics(J_eta,V);
            Eta+=Eta_dot*T;
            Visual.Draw(mode,t,T,position_list,pose_list,speed_list,angular_speed_list,poses);
        }
        Write_data_v(Times,filenames[0]);
        Write_data_vv(Eta_list,filenames[1]);
        Write_data_vv(Reference_Eta,filenames[2]);
        Write_data_vv(Tau_list,filenames[3]);
        while(true)
            Visual.Draw(0,t,T,position_list,pose_list,speed_list,angular_speed_list,poses);
    }
}
```

图 21 PARAMS 类核心函数 2

6.3 控制器模块

控制器模块中包含了外环控制器、内环控制器和推力分配系统，其输入期望位置、当前位置姿态、速度角速度、时间等信息，即可求解出当前时刻系统所受到的推进器的推力和力矩。如下图所示：第一步调用 mpc 类的 ComputeDesiredVelocity 函数求得期望的速度；第二部调用 s_vud_flc 类的 ComputeDesiredForce 函数以求得推进器的期望输入；第三步调用 Thrust_distribution 来分配 8 个推进器的转速；第四部调用 Generate_Force 函数来生成作用在 AUV 上的推力和力矩：

```
MatrixXd DesiredVelocity;
DesiredVelocity=mpc1.ComputeDesiredVelocity(t,T,t_max,current_Eta,current_V,reference_Eta);

Matrix<double,6,1> DesiredForce;
DesiredForce=s_vud_flc1.ComputeDesiredForce(T,current_V,DesiredVelocity);

Matrix<double,8,1> distribution;
distribution=Thrust_distribution(DesiredForce);

MatrixXd Force_Generated;
Force_Generated=Generate_Force(distribution);
```

图 13 控制器主要调用代码流程

下图为 MPC 主要的数据处理部分，通过先前的数据处理获得了 E、H、g 矩阵，以及 QP 问题所需要的约束。调用预先预先写好的 solveQP 函数，即可求得所求解出来的期望速度变化量 U_star，在取出其第一位元素（因为优化了 p，所以只需要计算 u,v,w,q,r，即前 5 个元素）。然后加上当前的速度，即获得了期望速度：

```
    E=PSI*x_k;
    H=THETA.transpose()*QQ*THETA+RR;
    g=E.transpose()*QQ*THETA;

    // 求解 QP 问题
    VectorXd U_star(5*control_horizon_Nc);
    solveQP(H, g,A_I,V_max0-V_T,-V_min0+V_T, U_max0,U_min0, U_star);
    U_star0<<U_star(0),U_star(1),U_star(2), 0 , U_star(3),U_star(4);

    return U_star0+current_V;
```

图 14 外环 MPC 预测控制器主要代码

下图为内环变论域 S-平面的 S_VUD_FLC 控制器，通过对速度误差和速度误差的导数进行处理，即可获得推进器的期望输入：

```
Eigen::Matrix<double, 6, 1>S_VUD_FLC::ComputeDesiredForce(
        double T,
        const Eigen::Matrix<double, 6, 1>& actual_velocity,
        const Eigen::Matrix<double, 6, 1>& desired_velocity){

    Matrix<double, 6, 1> DesiredForce;

    err=desired_velocity-actual_velocity;
    err_dot=(err-err_up)/T;
    x2_sum+=err_dot;

    for(int i=0;i<actual_velocity.rows();i++){
        double ys;
        double x1,x2;
        x1=err(i);
        x2=err_dot(i);
        ys=fs_x(x1/alpha_x(x1),x2/alpha_x(x2))/beta_x(x1,x2)+Ki*x2_sum(i);
        DesiredForce(i)=ys;
    }

    err_up=err;
    return DesiredForce;
}
```

图 15 内环控制器代码实现

推力分配部分代码如下所示：通过输入 6 个方向的期望输入，经过直接逻辑

推力分配方法，即可获得 8 个推进器的推力分配：

```
Matrix<double,8,1>Thrust_distribution(Matrix<double,6,1>& DesiredVelocity){

    double X,Y,Z,K,M,N;
    X=DesiredVelocity(0);
    Y=DesiredVelocity(1);
    Z=DesiredVelocity(2);
    K=DesiredVelocity(3);
    M=DesiredVelocity(4);
    N=DesiredVelocity(5);

    Matrix<double,8,1>distribution;
    distribution<<  X+Y+N,
                    X-Y+N,
                    X-Y+N,
                    X+Y-N,
                    Z-K-M,
                    Z+K-M,
                    Z-K+M,
                    Z+K+M;
    return distribution;
}
```

图 16 推力分配部分代码

以下是推力生成部分代码，目前有了 8 个推进器的推力分配，根据公式 61：$\tau = Bu_c$，输入 8 个推进器的推力分配，即可获得实际作用在 AUV 上的推力和力矩：

```
UUV_MODEL uuv0;
Matrix<double,6,1>Generate_Force(Matrix<double,8,1>&Thrust_distribution){
    Matrix<double,6,1> Force_Generated;
    Force_Generated=uuv0.B*Thrust_distribution;
    return Force_Generated;
}
```

图 17 推力生成部分代码

6.4 模型模块

模型模块包含了动力学模型和运动学模型，该模型输入控制器输出的力和力矩，即可解算出 AUV 在该力和力矩下所产生的位移。

下图为动力学模型代码，通过输入 AUV 上的力和力矩，以及水动力、物理特性矩阵等参数，就可获得实 AUV 实时的自身加速度。

```
Matrix<double, 6, 1>dynamics(Matrix<double, 6, 6>& M_,Matrix<double, 6, 6>& C_V,
                             Matrix<double, 6, 6>& D_V,Matrix<double, 6, 1>& g_Eta,
                             Matrix<double, 6, 1>& Tau,double tau_e,Matrix<double, 6, 1>& V){

    Matrix<double, 6, 1>V_dot;
    Matrix<double, 6, 1> tau_E;

    tau_E<<tau_e,tau_e,tau_e,tau_e,tau_e,tau_e;

    V_dot=M_.inverse()*(Tau+tau_E-g_Eta-D_V*V-C_V*V);
    return V_dot;
}
```

图 18 动力学模型代码

下图为运动学模型代码，输入 J 矩阵和速度/角速度 V，即可求得大地坐标系

下的加速度和角加速度。

```
Matrix<double, 6, 1> kinematics(Matrix<double, 6, 6>&
J_eta,Matrix<double, 6, 1>& V){
    Matrix<double, 6, 1> Eta_dot;
    Eta_dot=J_eta*V;
    return Eta_dot;
}
```

图 19 运动学模型代码

6.5 可视化模块

可视化模块包含了 AUV 的物理模型，实际的形状，8 个推进器的分布，并且通过输入期望轨迹和 AUV 的实时状态（时间、速度、角速度、位置、姿态），可以将 AUV 的状态实时可视化，其中需要绘制 AUV 自身的物理形状，推进器的分布，大地坐标系原点，以及期望的跟踪路径和实际的跟踪路径：

下图是绘制大地坐标轴的代码以及绘制轨迹的代码：通过调用 pangolin 库中的函数，以实现可视化，其中相机的视角此处不再赘述：

```
void VISUALIZE::Draw_coord(){
    glBegin(GL_LINES);
    glColor3f(1,0,0);   glVertex3d(0,0,0);    glVertex3d(0.5,0,0);
    glColor3f(0,1,0);   glVertex3d(0,0,0);    glVertex3d(0,0.5,0);
    glColor3f(0,0,1);   glVertex3d(0,0,0);    glVertex3d(0,0,0.5);
    glEnd();
}
void VISUALIZE::Draw_Trace(vector<Matrix<double, 5, 1>>&Trace){
    glColor4f(0.5, 0, 0.5,0.3);
    glBegin(GL_LINES);
    int di=20;
    for (int i = 0;i <= Trace.size()-2-di;i+=di) {
        glVertex3d(Trace[i](0),Trace[i](1),Trace[i](2));
        glVertex3d(Trace[i+di](0),Trace[i+di](1),Trace[i+di](2));
    }
    glEnd();
}
```

图 22 绘制大地坐标系以及轨迹

下图是可视化的集成函数，在该函数里通过 while 循环实现窗口的不断更新。清理画布，绘制 AUV，绘制坐标轴，绘制轨迹，接着更新循环遍历：

```
void VISUALIZE::drawScene_final(vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>> poses,
                pangolin::OpenGlRenderState &s_cam_follow,pangolin::View &d_cam_follow,
                pangolin::OpenGlRenderState &s_cam_overall,pangolin::View &d_cam_overall){
    while (pangolin::ShouldQuit() == false) {
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);

        drawScene(poses,s_cam_follow,d_cam_follow);
        drawScene(poses,s_cam_overall,d_cam_overall);

        uuv_model.Draw_UUV(poses);
        Draw_coord();
        Draw_Trace(params.Reference_Eta);
        pangolin::FinishFrame();
        usleep(100);   // sleep    5000 = 5 ms    dt秒=dt*1000 ms  = dt*1000*1000
    }
}
```

图 23 可视化集成代码